

Ubuntu・OS基礎 参照資料

用語集・コマンド早見表

Ubuntu Server 24.04 LTS / 公開Lab v1.0.0
JDU Cyber Security / 2026-09-25

Ubuntu・OS基礎 用語集・コマンド早見表

対象: Ubuntu Server 24.04 LTS / 公開Lab v1.0.0。本書は暗記表ではない。目的からコマンドを探し、必要なオプションと確認方法を本文で読み直すために使う。

用語集

日本語表記	English	意味
OS	operating system	ハードウェアを管理し、プロセス、ファイル、アクセス制御、通信などの共通機能を提供する基本ソフトウェア（システム）
カーネル	kernel	実行中OSの中核。CPU、メモリ、デバイス、プロセス、ファイル、ネットワークなどを管理する
ユーザー空間	user space	シェル、コマンド、サービスなどの通常プログラムがカーネルから分離されて動作する領域
ディストリビューション	distribution	Linuxカーネル、ツール群、パッケージ、初期設定、更新方針などを統合したOS体系。Ubuntuはその一例
コマンドライン	command-line interface (CLI)	文字列（テキスト）で指示と結果をやり取りする操作方法（インターフェース）
ターミナル	terminal	文字を入力し、結果を表示する画面・対話接続の環境
シェル	shell	入力された文字列を解釈し、コマンドを実行するプログラム。本演習環境では主にBashを使用する
ファイル	file	データ内容とメタデータを保持する、保存領域およびシステム表現の基本単位
ディレクトリ	directory	ファイル名や他のディレクトリ名を対応付け、階層構造を構築する特殊なファイル
パス	path	ディレクトリの階層構造を順にたどって対象の位置を指定する文字列
所有者	owner	ファイルやディレクトリのメタデータに記録された所有ユーザー
主グループ	primary group	ユーザーごとに基本として1件登録される主要グループ
補助グループ	supplementary group	共有資源へのアクセス権などのため、ユーザーが追加で所属するグループ
アクセス権（パーミッション）	permission	所有者（owner）、グループ（group）、その他（others）に対して読み取り（r）、書き込み（w）、実行（x）を許可する基本アクセス制御
プログラム	program	ストレージ（保存領域）に保存された実行可能な命令群および関連ファイル
プロセス	process	プログラムがメモリ上に読み込まれて実行されている状態。固有のPIDと実行資格情報を持つ
パッケージ	package	ソフトウェアを構成するファイル群、バージョン、依存関係、導入手順をまとめた配布単位
サービス	service	システムによって管理され、バックグラウンドで継続的に機能を提供するプログラムおよびその管理単位

日本語表記	English	意味
ユニット (unit)	unit	systemdがサービスなどを管理・制御するための設定単位
ソケット	socket	プロセスがカーネルのネットワーク通信機能を利用するための出入口
待受 (listen)	listen	クライアントからの接続リクエストを受け付け可能なソケットの状態
ループバック	loopback	同一ホスト内部へ折り返す仮想ネットワークインターフェース。IPv4の代表例は127.0.0.1
ポート	port	同一ホスト内で動作する複数の通信窓口（プロセス）を識別するための番号
ログ	log	プログラムやOSシステムが動作履歴、エラー、状態変化などを記録した情報
ローカル	local	SSHやSCPの接続・転送を開始する側の環境。本演習ではAWS CloudShellを指す
リモート	remote	SSHやSCPの接続・転送先となる遠隔サーバー環境。本演習ではUbuntu EC2を指す
認証	authentication	接続や操作を要求している主体が本人であるかを確かめること（誰であるかの確認）
認可	authorization	認証された主体に対して、対象リソースの操作を許可するか判断すること（何ができるかの決定）

必須コマンド

目的	コマンド・必要オプション	注意すべき点・誤解	操作後の確認
現在位置	pwd	プロンプトの表示文字列だけで判断せず、現在位置を確認する	出力されたパスとホスト名・ユーザー名を照合する
移動	cd PATH、cd ..、cd ~	相対パスはカレントディレクトリ（現在位置）に依存する	pwd
一覧	ls -la、ls -ld DIR	-l DIRはディレクトリ内の項目一覧を表示し、-ld DIRはディレクトリ自身の属性を表示する	必要に応じてstatやtreeを併用する
階層表示	tree PATH	階層構造は把握できるが、ファイル内容やアクセス権までは網羅しない	ls -l等と併用する
ディレクトリ作成	mkdir -p PATH	意図しない階層パスを誤作成しない	tree、ls -ld
コピー	cp SOURCE DEST	コピー元とコピー先の順序間違い、意図しない上書きに注意する	cat、sha256sum
移動・改名	mv SOURCE DEST	移動先への不用意な上書きに注意する	ls
削除	rm FILE	通常のGUIのような「ごみ箱」はない。演習課題では安全のためrm -rfを使用しない	実行前にpwdとlsで対象を確認し、実行後に不在を確認する
編集	nano FILE	意図したファイル名・パスで保存したか、未保存のまま終了していないか確認する	Ctrl+O、Enterで保存し、Ctrl+Xで終了後、catで内容を再確認する
内容表示	cat FILE、less FILE	バイナリファイルや極端に巨大なファイルを不用意にcatしない	lessはqで終了する
検索	grep [-F] PATTERN FILE	-nは行番号を付加する。課題の提出要件に行番号が含まれていない場合は付けない	出力された一致行と終了ステータスを確認する
末尾	tail -n N FILE	-nは出力行数を指定する（grep -nの行番号表示とは意味が異なる）	行数と順序を確認する
標準出力を保存	>、>>	>は既存内容を切り詰めて（空にして）上書きする。入力元と同じファイルを指定すると内容を失う	cat、wc -l
コマンド接続		標準出力だけが渡され、標準エラー出力（stderr）はパイプを流れない	パイプで連結する前に、各コマンドを単独でも実行して動作を確認する
ユーザー/グループ	id [USER]、groups [USER]	アカウントの登録情報と、起動済みセッションが保持する資格情報を区別する	新しいセッション（ログインシェル）を開始した上でidを実行する
アカウントDB	getent passwd USER、getent group GROUP	グループエントリの末尾一覧には、そのグループを主グループとしているユーザーが含まれない場合がある	id USERと併用して確認する
ユーザー切り替え	sudo su - USER、exit	シェルが多重に入れ子になる点に注意する。演習用一般ユーザーにはsudo権限がない	切り替えのたびにid -unやpwdで現在状態を確認する
1コマンドだけ別ユーザー	sudo -u USER -- COMMAND	rootでの実行成功を、対象ユーザーでも成功する証拠とみなさない	実行直後にecho \$?で終了ステータスを確認する
グループ追加	usermod -aG GROUP USER	-aなしで-Gを指定すると、記載しなかった既存の補助グループから除外される	id USERを実行し、新しいセッションで確認する
グループ削除	gpasswd -d USER GROUP	ユーザー名とグループ名の指定順序に注意する	id USER
所有者/グループ変更	chown OWNER:GROUP PATH	再帰的変更（-R）による影響範囲に注意する	stat -c '%U:%G %a %n'

目的	コマンド・必要オプション	注意すべき点・誤解	操作後の確認
モード変更	chmod MODE PATH	安易に777を設定しない。ファイルとディレクトリで実行権限 (x) の意味が異なる	statや別ユーザーでのtestコマンドで確認する
プロセス一覧	ps -e -o ..., ps -p PID -o ...	教科書の例示PIDをそのまま入力しない	ユーザー名、コマンド名、PIDが一致していることを照合する
シグナル送信	kill -TERM PID	PID変数が空、0、負値、1 (PID 1) でないことを確認する。PIDに対してpkillを使わない	psやサービスの稼働状態で終了を確認する
パッケージ情報・導入	apt show NAME、sudo apt update、sudo apt install NAME	apt update (一覧更新) と apt upgrade (本体更新) の違いを理解する。演習環境で無関係な全体更新を行わない	command -v、dpkg-query
サービス設定	systemctl cat UNIT	演習用のユニットファイルを直接編集しない	User、WorkingDirectory、ExecStartの各項目を確認する
サービス状態	systemctl status/start/stop	active (起動中) であることだけで、機能全体が正常に動作していると判断しない	is-active、MainPID、curlなどで多角的に確認する
起動時自動起動	systemctl enable/disable UNIT	即時起動 (start/stop) と自動起動設定 (enable/disable) は別軸である	is-enabled
Main PID取得	systemctl show -p MainPID --value UNIT	Main PIDが0、空、または再起動前の古い値でないか注意する	ps -p PID
ソケット状態	sudo ss -lntp	アドレス、ポート番号、プロセスPIDをすべて照合する	サービスのMain PIDと比較する
HTTP疎通	curl -i URL、必要時--max-time	TCP接続の成功、HTTPステータスコード、応答本文の内容はそれぞれ個別に確認する	ステータスライン、本文、終了ステータスを確認する
ジャーナルログ	sudo journalctl -u UNIT --no-pager -n N	過去の古い起動ログと、現在の実行ログを混同しない	指定されたリクエストパスが記録されているか確認する
SSH接続	ssh HOST、ssh HOST 'COMMAND'	ホストエイリアスとDNSホスト名の違い、クォートによるローカル/リモートでの変数展開の違い、秘密鍵の保護に注意する	id -un、hostname、pwdで接続先環境を確認する
SCPファイル転送	scp SOURCE DEST	転送元 (source) と転送先 (destination) の取り違え、不用意な上書きに注意する	両環境でsha256sumを比較し、リモート側でstatを確認する

コマンドの読み方

コマンドは「名前・オプション・対象」に分けると読みやすい。例えば、`tail -n 4 practice.log`では、`tail`が実行するプログラム、`-n 4`が行数を指定するオプションと値、`practice.log`が読む対象である。空白で区切られた部分を順に読む。マニュアルに出るFILEやPATH、Nなどの大文字は、実際のファイル名・パス・数値に置き換える説明用の記号である。

オプションは動作を変える指定である。短い形式は-uのように1文字、長い形式は--userのように単語で表すことが多い。ただし、すべてのコマンドが両方に対応するわけではない。`find`の-nameのように、ハイフンが1つでも単語を使う指定もある。綴りから推測した後、必ずそのコマンドの説明で意味を確認する。

英語の意味と結び付けて覚える

次の英語は、コマンドを覚えるための手掛かりである。すべてを歴史的な正式名称・略語の由来として断定するものではない。`find`や`tail`は、英単語そのものを使っている。

コマンド	覚えるための英語	読み方・用途
pwd	print working directory	現在作業しているディレクトリを表示する
cd	change directory	作業するディレクトリを変える
ls	list	名前の一覧を表示する
mkdir	make directory	ディレクトリを作る
cp	copy	コピーする。コピー元、コピー先の順に指定する
mv	move	移動する。同じディレクトリ内なら名前変更にも使う
rm	remove	指定したファイルを削除する
cat	concatenate (連結する)	複数ファイルの内容を連結して表示する。1ファイルの表示にも使う
tree	tree (木)	枝のような形でディレクトリ構成を表示する
find	find (探す)	ディレクトリをたどり、条件に合うファイルやディレクトリを探す
grep	global / regular expression / print (g/re/pに由来)	ファイルの中の行を検索する。ファイル名を探すfindと区別する
tail	tail (末尾・しっぽ)	ファイルの末尾の行を表示する
id	identity / identifier (識別情報・識別子)	ユーザーやグループの識別情報を表示する
chown	change owner	所有者を変更する
chmod	change mode	アクセス権のモードを変更する
ssh	Secure Shell	別のコンピューターへ安全な通信で接続する

id -unを分解して読む

idはユーザーやグループの識別情報を表示する。オプションなしのidはUID、GID、所属グループなどをまとめて表示する。

```
id
id -u
id -u -n
id -un
```

-uは--userに対応し、現在の実効ユーザーIDだけを表示する。-nは--nameに対応し、番号の代わりに名前を表示する。したがってid -u -nとid -unは同じ意味で、今回のUbuntu接続直後ならssm-userを表示する。

id -nだけでは、どのIDを名前に変えるか決まらない。-nは-u、-g、-Gのいずれかと組み合わせる。id -gnは実効グループ名、id -Gnは所属する全グループの名前を表示する。英語を手掛かりにするだけでなく、組み合わせの条件まで確認する。

同じ文字でも、コマンドによって意味が変わる

実行例	オプションの英語・意味	実行結果
id -un	-u=user、-n=name	現在の実効ユーザー名
grep -n 'WARN' practice.log	-n=--line-number	一致した行に行番号を付ける
tail -n 4 practice.log	-n=--lines。続く4が行数	最後の4行を表示する
ls -la	-l=long形式、-a=all	詳細表示で、隠しファイルも含める
mkdir -p logs/archive	-p=--parents	必要な親ディレクトリも作る
grep -F 'WARN' practice.log	-F=--fixed-strings	正規表現ではなく文字列として検索する
chown -R USER DIR	-R=--recursive	DIRの中まで再帰的に所有者を変更する

id -unのようにまとめられる短いオプションもあるが、すべてをまとめられるとは限らない。tail -n 4のように値を取る指定や、find -nameのような単語の指定は、今回の例の形で入力する。

保存記号はコマンドのオプションではない

```
grep 'WARN' ../logs/practice.log
grep 'WARN' ../logs/practice.log > warnings.txt
```

1行目は結果を画面に表示する。2行目の>は、シェルが標準出力の行き先をwarnings.txtへ変える記号であり、grepのオプションではない。保存できたらcat warnings.txtで内容を確認する。>>は追記する記号である。

findを段階的に読む

treeはディレクトリ全体の形を見て探すのに向いている。findは条件を指定して対象を絞り込むのに向いている。P1では、まずtreeで構成を観察し、次にfindで.tmpファイルを検索し、確認したパスをrmへ指定する。

検索開始位置・条件・表示に分ける

```
find . -type f -name '*.tmp' -print
```

このcommandは「現在地以下をたどり、通常のファイルで、名前が.tmpで終わるもののパスを表示する」と読む。

部分	意味	覚え方・注意
find	検索するコマンド	find=探す
.	検索を始める位置	現在のディレクトリ。配下もたどる
-type f	通常のファイルに限定する条件	type=種類、f=file。ディレクトリなら-type d
-name '*.tmp'	名前がパターンに一致する条件	name=名前。ファイル内容は検索しない
-print	一致したパスを表示する動作	print=出力する。ファイルを印刷する意味ではない

-typeと-nameは正確には「テスト（条件）」、-printは「アクション（動作）」である。間に演算子を置かない複数の条件は、通常「両方を満たす」というANDとして評価される。今回の-type f -name '*.tmp'なら、名前が一致してもディレクトリは表示しない。

-printなどの動作を省いた単純な検索では、通常、検索結果のパスが表示される。ここでは何をするか読み取れるように-printを明示する。

小さい指定から条件を足す

P1で、次を1行ずつ実行して表示の変化を比べる。

```
cd ~/jdu-lab/p1/practice01
find . -print
find . -type f -print
find . -type f -name '*.tmp' -print
```

最初は開始位置自身と配下の項目を表示し、次は通常のファイルだけを表示し、最後は名前が`.tmp`で終わる通常のファイルだけを表示する。1段階ずつ条件を足すと、各指定が何を絞っているか理解しやすい。

削除前のP1では、最後の検索で次の2つが表示される。表示順は保証されない。

```
./staging/training.conf.tmp
./staging/practice.log.tmp
```

すでに手順6で削除した場合は、この検索で何も表示されない。その場合も正常であり、確認のためだけに課題をリセットする必要はない。

なぜ引用符で囲むのか

`'*.tmp'`の`*`は、0文字以上の任意の文字列に一致する。このパターンを引用符で囲むと、シェルが現在地の名前に展開せず、`find`へそのまま渡す。`find`が各階層のファイル名に対して一致を調べる。引用符を外すと、現在地に同じパターンの名前がある場合、`find`を実行する前に引数が変わってしまう。

`-name`は途中のディレクトリを含まない名前に対して調べる。`training.conf.tmp`を探すなら`-name '*.tmp'`を使う。パス全体に条件を付けたい場合は`-path`を使う。これは正規表現ではなく、ファイル名パターンの検索である。

よく使う条件を必要なときに追加する

指定	英語の手掛かり	意味
<code>-type d</code>	directory	ディレクトリだけを検索する
<code>-iname '*.log'</code>	大文字・小文字を区別しないname条件	名前の大文字・小文字を区別せず検索する
<code>-path './staging/*'</code>	path	パス全体がパターンに一致するものを検索する
<code>-maxdepth 2</code>	maximum depth	開始位置を深さ0として、深さ2までたどる
<code>-mindepth 1</code>	minimum depth	開始位置自身を結果に含めず、深さ1以降を評価する
<code>-empty</code>	empty=空	空の通常ファイル、または空のディレクトリに一致する
<code>-user ssm-user</code>	user	指定したユーザーが所有する項目に一致する

`-maxdepth`や`-mindepth`は、検索の範囲を制御する指定なので、開始位置の直後に置くと読みやすい。例えば、次は現在地から2階層までで通常のファイルを探す。

```
find . -maxdepth 2 -type f -print
```

条件と削除に分ける

P1では`find`は検索と表示だけに使う。検索結果を確認してから、`rm`に具体的なパスを指定する。

```
rm staging/training.conf.tmp staging/practice.log.tmp
tree
find . -type f -name '*.tmp' -print
```

treeで構成を見直し、最後のfindで該当するパスが表示されないことを確認する。findにはdeleteや-execのように対象を変更したり別のコマンドを実行したりする動作もあるが、今回の練習では使わない。例を読み取れるまでは、検索条件と結果の確認を分けて進める。

grepでファイルの中の行を探す

grepは、指定したパターンに一致する行を取り出す。名前はUnixの行エディターedのg/re/p（global / regular expression / print）に由来する。まずは「内容を検索し、一致する行を表示する」と覚える。findはファイル名や種類などを探し、grepはファイルの内容を探す。

```
grep 'WARN' ../logs/practice.log
```

grepがコマンド、'WARN'が探すパターン、../logs/practice.logが読むファイルである。既定では大文字・小文字を区別し、パターンを基本正規表現として解釈する。今回のWARNは、その文字列を含む行に一致する。表示されるのは一致した単語だけでなく、行全体である。

指定	対応する英語・長い形式	意味
-n	--line-number	一致した行に元の行番号を付ける
-i	--ignore-case	大文字・小文字を区別しない
-F	--fixed-strings	パターンを正規表現ではなく文字列として扱う
-v	--invert-match	一致しない行を取り出す
-c	--count	一致した行の数を表示する。単語の出現回数ではない

例えば、grep -n 'WARN' ../logs/practice.logはWARN行に行番号を付ける。P1やM1で「行番号を付けない」と指定された提出ファイルでは、保存時に-nを使わない。grep -F 'WARN' ../logs/practice.logなら、検索する内容を通常の文字列として指定できる。

観察してから保存する

```
grep 'WARN' ../logs/practice.log
grep 'WARN' ../logs/practice.log > warnings.txt
cat warnings.txt
```

まず抽出する行が正しいか画面で確認し、その後、同じ検索の結果を別ファイルへ保存する。catで保存した内容を読み直す。>を付けた実行で画面に結果が表示されなくても、保存先へ出力されていれば正常である。

何も表示されない場合は、一致する行がないこともある。grepの終了ステータスは、通常、0が一致あり、1が一致なし、2がエラーである。ファイルが存在しない場合のエラー表示と、「一致する行がない」状態を区別する。

tailで末尾の行を読む

tailは英語で「末尾・しっぽ」を意味する。ログの最後にどのような出来事が記録されたか確認するときに使う。オプションを付けけないtail FILEは、既定で最後の10行を表示する。行数がそれより少なければ、存在する行だけを表示する。

```
tail -n 4 ../logs/practice.log
```

tailがコマンド、-n 4が表示する行数、../logs/practice.logが読む対象である。-nは長い形式--linesに対応し、次の4を行数として受け取る。grep -nの「行番号を付ける」とは意味が異なる。

指定	対応する英語・長い形式	意味
-n 4	--lines=4	最後の4行を表示する
-n 5	--lines=5	最後の5行を表示する
-f	--follow	終了せず、新しく追加された内容も表示する

tailは最後の行から逆順に表示するコマンドではない。選ばれた末尾の行を、ファイル内と同じ順序で表示する。P1では最後の4行、M1では最後の5行を指定する。

観察してから保存する

```
tail -n 4 ../logs/practice.log
tail -n 4 ../logs/practice.log > recent.txt
cat recent.txt
```

最初の実行で行数と順序を観察し、次の実行で保存する。P1ではrecent.txtの内容が最後の4行と一致することを確認する。

ログを見続ける補助的な操作にはtail -f FILEがある。これは入力を待つ状態になるため、終了するときはCtrl+Cを押す。提出ファイルを作る今回の手順では、終了するtail -n N FILEを使う。

意味がわからない指定を調べる

```
id --help
find --help
man find
help cd
```

--helpは多くの外部コマンドの短い説明、manは詳しいマニュアル、helpはBash内蔵コマンドの説明を読む手段である。manの画面はqで終了する。Labでmanや個別のマニュアルが未導入なら、--help、helpまたは次の公式資料を使う。

- [GNU Coreutils: id](#)
- [GNU Grepマニュアル](#)
- [GNU Coreutils: tail](#)
- [GNU Findutilsマニュアル](#)

調べる順序は、(1) コマンドの役割、(2) オプションの意味、(3) 続く値、(4) 操作する対象である。AIに相談する場合も、find . -type f -name '*.tmp' -printを各部分に分けて説明してもらい、--helpやマニュアルと照合する。

授業内で使うが暗記不要

uname -r、hostname、find、touch、stat、namei -l、test -r/-w、echo \$?、printf、command -v、dpkg-query、dpkg -S、sha256sum、readlink、tr、/proc/PID/。

目的と出力の読み方を本文で確認し、必要なときにマニュアル (man/help) や本早見表を参照する。

後続単元で扱うコマンド

ip、ping、dig、tcpdump、mount、df、psql。本書では関連性を示すにとどめ、詳細な設定や操作はネットワーク、ストレージ、DNS、データベースの各単元で扱う。

本書の対象外

カーネルのビルドやデバッグ、CPU命令の内部処理、メモリページ置換アルゴリズム、ディスクI/Oスケジューラの操作、ファイルシステムの作成・パーティション変更、外部ホストへのポートスキャン、SSHサーバーの設定変更。